Project Stage 2

Assignment Group17

Unikey	SID
xiwu3008	500078406
kwan0569	540979271
yzan0977	540945380

Group Component 1

1. Topic and Research Question

The research question if this project is: Is it possible to effectively predict whether an individual has diabetes based on basic attributes, healthy status and lifestyles? According to *Diabetes Atlas 11th Edition* of International Diabetes Federation in 2025, there are over ti.4 million people died as a result of diabetes in 2024, which accounts 9.3% of global death from all causes. In addition, the diabetes is regarded as an incurable disease, nevertheless, through effective management and treatment, the symptoms of diabetes can be significantly improved especially in the early stage. While predicting the risk of diabetes helps to identify whether an individual is ill in advance, carry out medical intervention as early as possible, and reduce and avoid the late diagnosis and risk of death, therefore, the research on prediction of diabetes is significant in medicine and public health domain, which relates various of stakeholders. For the public, they can identify the possibility of having diabetes early through their own living habits and health conditions. If there is a risk of developing diabetes, they can seek medical intervention in time to better control the development of diabetes at an early stage. For medical professionals, it is possible to better identify potential diabetic patients and provide them with more effective and personalized diagnostic and treatment plans. And for public health departments, accurate diabetes prediction can identify high-risk populations, improve the efficiency of prevention and early screening, better promote diabetes prevention and control measures, and optimize resource allocation.

2. Datasets

There are X entries and Y columns in the dataset, consisting with A, B, C, and D features. It is worth noting that only the information of adults (aged over 18) was collected, and considering that diabetes is more common in adults, so the prediction of the models is only applicable to adults.

The target variable is **Diabetes**, with two possible values: *'Diabetes'* and *'No'*, indicating whether an individual is diabetic or not, respectively. Among the samples, 85.8% are labelled *'No'*, and only 14.2% are labelled *'Diabetes'*, resulting in an approximate 6:1 class ratio. This represents a moderate class imbalance, which can lead to the model being unable to effectively learn the characteristics of the minority classes. Consequently, the model may become biased toward the majority class and cause serious misguidance when evaluated using common metrics such as accuracy. For instance, a model that predicts all samples as *'No'* would still achieve an accuracy of 85.8%, despite failing to detect any diabetic cases. To address this issue, techniques such as **undersampling** and **oversampling** can be employed, randomly reducing samples from the majority classes and synthesizing new samples from the minority classes. Additionally, since the imbalance is moderate, the tuning parameters, such as class weights, can be adjusted at the model level to place greater emphasis on the minority class during training. It is also essential to adopt more informative and fair evaluation metrics, such as **PR-ROC** and **F1-score**, which provide a fairer assessment of model performance under imbalanced conditions.

In addition, the distribution of feature **BMI** is highly unbalanced. Aside from a large number of extreme values, the majority of BMI values are above 25, indicating that most of individuals fall into overweight category according to the BMI classification of NSW government. This suggests potential selection bias in the dataset, which could impair the ability of model to accurately predict outcomes for individuals with lower BMI. As a result, the model may perform poorly in real-world deployments, leading to reduced reliability and generalizability. While resampling is the ideal solution to address this bias, it can be difficult to implement. An alternative approach is sample weighting, where the weight of samples with lower BMI is increased, encouraging the model to focus more on this subset, but it is essential to evaluate the performance of the model on different samples is more balanced.

3. Set up

3.1 Modelling agreements

In the second stage, our group selected the Diabetes Health Indicators Dataset. In this dataset, what we chose was the attribute of whether people have diabetes. This attribute contains three categories: No (No diabetes), Diabetes (Diagnosed diabetes) and Prediabetes. Because only the attribute of whether people have diabetes is focused on, “Diabetes” is simplified to a binary classification problem. Therefore, we only predict “Diabetes” and “No”, excluding “Prediabetes”. Set "No" to “0” and "Diabetes" to “1”.

Meanwhile, the category of Diabetes in this dataset is highly imbalanced. "No diabetes" accounts for the majority (more than 84%), while truly "diabetic" patients only account for approximately 14%. Then, we will select the three indicators of F1-score, ROC-AUC, PR-AUC, recall and precision to measure whether the predictive model is successful.

Among these three indicators, F1-score refers to the harmonic meaning between Precision and Recall, and Precision measures the proportion of samples predicted to be in the positive class that are actually in the positive class. Similarly, Recall measures the proportion of all positive class samples that are correctly predicted as positive classes. The reason for using F1-score is that in the dataset studying whether people have diabetes, the number of people with diabetes (positive category) is much less than that of people without diabetes (negative category), resulting in category imbalance. Because the samples of diabetes patients are in the minority Class (Positive Class), if the model only learns to predict that the majority are "No diabetes", it can achieve

high Accuracy, but this is not practical. Therefore, in this case, using F1-score can conduct a comprehensive assessment when the data is imbalanced.

ROC- AUC refers to Area Under the Curve, which is defined as the area enclosed by the coordinate axes under the ROC curve. Therefore, this area will not be greater than 1. Also, since the ROC curve is generally located above the straight-line y=x, the value range of AUC is between 0.5 and 1. That means if the ROC curve of the model is significantly higher than the diagonal, it indicates that it has a good discriminative ability. The ROC curve has two dimensions. The horizontal axis is False positive rate (FP rate), and in this dataset, it represents the proportion of people who misjudge "No diabetes" as "Diabetes". The vertical axis is The positive rate (TP rate), in this dataset, it represents the proportion of those who correctly identify "Diabetes". And the lower left corner (0,0) indicates that the classifier predicts all the test sets as negative cases and does not perform positive class predictions. So, both TP rate and FP rate are 0; The upper right corner is (1,1), indicating that the classifier predicts all the test sets as positive examples. Therefore, both the TP rate and the FP rate are 1. Generally speaking, the closer the points in the ROC are to (0,1), the better the representation (TP rate is higher and FP rate is lower). In the model, if the AUC is between 0.8 and 0.9, it indicates that the model can effectively distinguish the population with "Diabetes" from that "No diabetes". The reason for using ROC- AUC is that AUC is not easily affected by class imbalance and is suitable for measuring the overall model ranking ability.

PR-AUC refers to the area under the Precision-Recall Curve and is used to measure the performance of the model under different classification thresholds. Unlike ROC-AUC, PR-AUC focuses on the relationship between “precision” and “recall”. In PR-AUC, “precision” represents the proportion of people who are actually diagnosed with diabetes by the model; "Recall" represents the proportion of all diabetic patients identified. Because the proportion of people with diabetes in the dataset is relatively low, PR-AUC can reflect more clearly than ROC-AUC whether the model has truly identified the positive class. PR-AUC pays more attention to the trade-off between accuracy and recall rate. Accuracy measures the proportion of instances that the model predicts as positive samples but are actually positive samples, while recall measures the proportion of instances that the model correctly predicts as positive samples among all instances that are actually positive samples. In imbalanced datasets, or when false positives receive more attention than false negatives, the trade-off between accuracy and recall rate is particularly important. Based on the above situation, PR-AUC is applicable to extremely imbalanced datasets.

3.2 data division

Before data division, the attributes to be predicted need to be listed as the label set y, and the rest are the feature set x. In this division, "Diabetes" is the label set. Through the observation of the label set, it can be found that there is an imbalance in the ratio of positive and negative samples (≈ 14:86) in the dataset. Direct random division may lead to some subsets not containing a few categories or distribution distortion. Stratified Sampling ensures that the label distribution in each training set and test set is consistent with the overall distribution by taking samples respectively at the original proportion of label y within each fold. Meanwhile, we also used cross-validation to reduce the evaluation fluctuations caused by the contingency of a single partition. In the specific code implementation, we use the StratifiedKFold function in scikit-learn and set n_splits=5, shuffle=True, and random_state=42 to ensure the stratification effect, randomness and reproducibility. This function will first apply random shuffling to the entire data set to eliminate the potential deviation caused by the original input order. Then the shuffled data is equally divided into k=5 parts, with approximately N/5 records in each part; After that, five training-test cycles will be conducted: For the i-th fold: The i-th copy will be taken as the "test set", and the remaining four copies will be combined into the "training set". Meanwhile, within each fold, samples will be drawn respectively according to the original proportion of label y to ensure that the label distribution in each training set and test set is consistent with the overall. Finally, save the training set and test set of each fold respectively as CSV files (train_fold_i.csv, test_fold_i.csv), which is convenient for subsequent batch training and comparison of the model.

By using stratified sampling and cross-validation, there are the following several advantages:

- Evaluating stability: 5-fold cross-validation enables each sample to participate in both training and testing, reducing the evaluation fluctuations caused by the randomness of a single partition.
- Representativeness guarantee: Stratified sampling ensures that minority categories (i.e., the number of people with diabetes) are adequately represented at each fold, avoiding performance distortion of the model on certain test sets due to unbalanced label distribution.
- Full utilization of data: On the limited diabetes dataset, the multi-fold training-test cycle maximizes the utilization of all available samples.

Individual Component – xiwu3008

1. Predictive model

1.1 Model description

The model employs eXtreme Gradient Boosting (XGBoost), an ensemble learning technique based on the gradient boosting framework for prediction by constructing multiple decision trees. Therefore, XGBoost largely inherits the assumptions of decision tree models, while also introducing its own distinctive mechanisms and considerations. Firstly, as a supervised learning algorithm, XGBoost predicts the target variable by learning patterns from input features. It therefore assumes the existence of a statistical or logical relationship between the features and the target. The objective of this research is to predict whether an individual has diabetes through features such as basic attributes, health status and lifestyles. Based on medical common sense and exploratory data analysis in Project Stage 1, there is clear evidence of a meaningful relationship between these features and the presence of diabetes. Furthermore, XGBoost mainly operates on numerical features. Although the dataset contains many categorical features, they have been safely encoded as numerical values and take into account the differences between nominal (e.g. *Sex*) and ordinal (e.g. *Education*), avoiding the introduction of misleading ordinal relationships. This encoding strategy ensures to meet the assumptions in the underlying technology.

Although XGBoost, as a non-parametric model based on decision trees, does not impose strict assumptions on data distribution, it does rely on the assumption that samples are independently and identically distributed. This means that each data point is independent of each other without dependency and drawn from the same probability distribution. However, considering that the data was sourced from interviews in 51 states of the United States, that is, all samples originate from the same national context, this assumption might be slightly violated due to potential regional similarities or bias. Additionally, while XGBoost is relatively robust to outliers, an excessive number of them can lead to structural biases in the model. However, the dataset contains several extreme values, which could potentially influence model performance.

Taking BMI as an example, the box plot from Stage 1 reveals a significant number of extreme values above 40 (BMI above 40 is recognized as extreme obesity in medical domain and also recognized as an outlier from a statistical perspective according to the box plot). Excessive outliers damage the generalization ability of the model. The tree structure does not split meticulously within the normal value range but allocates disproportionate attention to handling these extreme samples, resulting in the model being dominated by outliers, overfitting extreme values while making inaccurate predictions for mainstream samples.

In conclusion, the research on diabetes classification and prediction involves multiple variables with complex, non-linear relationships. Despite some limitations, XGBoost is a suitable choice due to its ability to handle non-linearity, capture feature interactions, and deliver high predictive accuracy.

1.2 Model algorithm

The XGBoost algorithm is a gradient boosting tree method that serves as the core technology of the model. It integrates multiple weak learners (classification and regression tree, CART for short) into a strong ensemble model. During training, XGBoost iteratively builds new trees to fit the residual errors of the previous model by optimizing a objective function. Each new tree focuses on correcting the prediction errors of its predecessors, thereby gradually improving the overall model performance. The final prediction is obtained by a weighted summation of all the individual tree outputs, leading to a more accurate and robust result. The objective function in XGBoost consists of two components: a loss function and a regularization term. The loss function $\boldsymbol{l}$, measures the discrepancy between the predicted and true values, while the regularization term $\boldsymbol{\Omega}$ penalizes model complexity to prevent overfitting. The performance of model is determined by the bias and variance. Specifically, the loss term represents the bias, while the regularization term controls the variance. These two components must be balanced to avoid underfitting from high bias or overfitting from high variance. Thus, minimizing the overall objective function L is essentially a bias-variance trade-off.

$$Obj = \sum_{i=1}^{n} l(y_i, \hat{y}_i) + \sum_{i=1}^{t} \Omega(f_i)$$

The subtree construction strategy in XGBoost is consistent with that of CART, both follow a greedy approach by evaluating the gain at each potential split and selecting the features that maximizes the gain for splitting. Furthermore, XGBoost enhances traditional gradient boosting by applying a second-order Taylor expansion to approximate the objective function. Consequently, the gain calculation depends on both the first derivative Gradient ($\boldsymbol{g_i}$) and the second derivative Hessian ($\boldsymbol{h_i}$). As a result, the objective function of XGBoost is ultimately in the following form:

$$Obj^{(t)} \simeq \sum_{i=1}^{n} [g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i)] + \Omega(f_t)$$

XGBoost offers a wide range of hyperparameters that can be tuned to improve model performance. In this study, the following key hyperparameters were primarily utilized:

- ***eta***: known as learning rate, controls the contribution of each tree to the final model.
- ***max_depth***: specify the maximum depth of each tree, control the complexity of the model.
- ***n_estimators***: specify the number of trees (boosting round) trained in total.

- ***colsample_bytree***: specifies the fraction of features to be randomly sampled for each tree, increase randomness.
- ***subsample***: specify the proportion of data subsets used in each round of training, control overfitting.
- ***scale_pos_weight***: specify the weights to positive class and help the model pay more attention to the minority class.
- ***eval_metric***: specify the evaluation metrics during training, such as aucpr (PR-AUC).

```
1.      Initialize base prediction F(x)
2.      For t=1 to n_estimators:
            2.1.  Compute gradients (g_i) and Hessians (h_i) (by scale_pos_weight)
            2.2.  Construct a new tree F_t(x) using (g_i, h_i):
                      if depth < max_depth:
                          Compute Gain
                          Choose split with highest Gain
            2.3.  Update model: F_t(x) = F_{t-1}(x) + eta * F_t(x)
            2.4   Evaluate performance (using eval_metric)
3.      Output final model:
            F(x) = F_0(x) + eta * F_1(x) + eta * F_2(x) + ... + eta * F_t(x)
4.      Make predictions using F(x)
```

Figure I - Overall Execution of Algorithm

In addition, XGBoost released version 2.1.0 on June 20, 2024, including many important improvements, such as support for the DataFrame data format. This update allows XGBoost to directly process DataFrame without manual encoding or internal conversion to C++ structures, improving performance and reducing memory usage. For the dataset used in the study, which contains many categorical variables, and this recent advancement simplifies data preprocessing, reduces manual workload, and helps preserve data integrity by avoiding the unintended introduction of ordinal relationships on nominal variables.

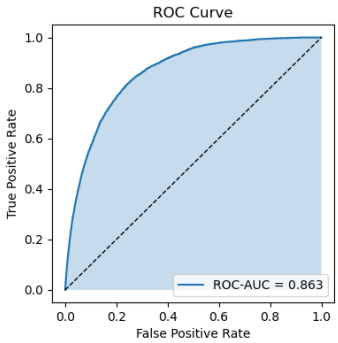
1.3 Model development

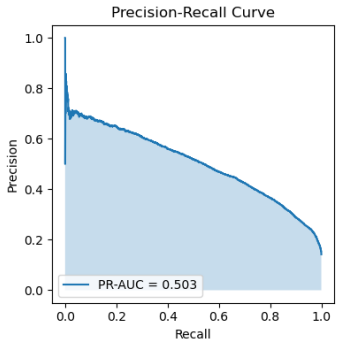
The model is built using XGBoost, combined with feature engineering and class imbalance adjustment. Hyperparameters are optimized using grid search and repeated stratified cross-validation, with PR-AUC as the main evaluation metric. First, considering the right-biased distribution and long tail of BMI in the research data, new features were created by grouping BMI values and assigning weights. The weights were calculated through the inverse of group sample sizes, followed by minimum normalization and clipping. Both the original BMI feature and the new weighted feature will be used to train models and evaluated to identify the version with better performance. Secondly, convert categorical variables to `pandas.Categorical` and set the `enable_categorical=True` to optimize memory usage and speed up training. The dataset in this study contains about 200,000 samples, and this processing greatly enhances memory efficiency and training speed. Then, the Stratified strategy and `train_test_split` function is adopted to divide the data into the test and training set at a ratio of 1:4. Because there is a problem of class imbalance in the data set, and the strategy can ensure that the distribution of the target variable remains consistent between the training and test set, thereby evaluating the model performance more fairly and accurately. Afterwards, hyperparameter tuning is performed using `GridSearchCV` and `RepeatedStratifiedKFold`. Among them, `GridSearchCV` can exhaustively search all parameter combinations and evaluates each using Stratified Cross-Validation. The combination with the best average performance is selected according to the specified metric (PR-AUC). This method can provide a comprehensive search to find the global optimal parameters within the defined range and reduce the negative impact brought by class imbalance. And after multiple practical comparisons, models tuned with grid search consistently outperform those using default or manually set parameters. The optimal parameter combination is ***colsample_bytree=0.8, eta=0.1, max_depth=5, n_estimators=50,*** and ***subsample=0.8***. This combination introduces randomness in both features and samples, helping to improve the model's generalization. At the same time, the lightweight model reduces model complexity while maintaining strong predictive performance. Finally, the model is implemented using the `XGBClassifier()` function from the **xgboost** library, which offers extensive flexibility in configuring hyperparameters. In addition to the parameters mentioned above, special attention is given to setting the `scale_pos_weight` parameter, defined as the ratio of the number of negative samples (class 0) to positive samples (class 1) in the training set. As previously noted, the proportion of the positive class in the research data is too small, while this parameter helps the model give more attention to it. By assigning greater weight to the minority class, it enhances the model's sensitivity to positive cases and mitigates the risk of biased predictions and poor generalization performance.

2. Model Evaluation and Optimization

2.1 Model evaluation

Research data on diabetes are similar to most medical-related datasets, showing a significant class imbalance because although the global prevalence of diabetes is steadily rising, it still represents a minority class within the population. Considering such data characteristics, model performance was comprehensively evaluated and interpreted using PR-AUC, ROC-AUC, precision, recall and F1-score. And given that the research question is whether it is possible to effectively predict whether an individual has diabetes based on some information, both precision and recall are important. Therefore, a default threshold of 0.5 is used: if the predicted probability is 0.5 or higher, the sample is classified as positive (class 1, diabetes); otherwise, it is classified as negative (class 0, no diabetes).





The ROC curve illustrates the model's performance across all classification thresholds by plotting the True Positive Rate (TPR) against the False Positive Rate (FPR). The ROC-AUC quantifies the model's ability to rank predictions, reflecting the probability that a randomly chosen positive sample is assigned a higher score than a randomly chosen negative one. For example, an ROC-AUC of 0.863 indicates that the model correctly ranks a positive case above a negative case 86.3% of the time. As shown in the figure on the left, the ROC curve quickly approaches the top-left corner, suggesting that the model maintains a low false positive rate across most thresholds. Therefore, despite the class imbalance, the model demonstrates a strong ability to distinguish between high-risk and low-risk individuals. In contrast, PR-AUC evaluates the model's ability to identify the minority class (positive cases). The PR-AUC in the left figure is much lower (0.503), indicating that while ensuring a high recall, the precision rate is very limited. However, in the scenario of class imbalance, a random model would generate a PR-AUC close to the proportion of the positive class (14%). Therefore, 0.563 is actually significantly well above random, but there is still leaves considerable room for optimization.

The classification report (Figure II – Classification Report of XGBoost Model) further highlights the trade-off. For the negative class, both precision (0.961) and recall (0.756) are high, indicating strong performance in identifying non-diabetic cases. The high F1-score (0.847) suggests the model maintains a good balance between false positives and false negatives. However, the classification results for the positive class align with the PR-AUC findings. While the model correctly identified 81.7% of diabetes cases (recall), 64.3% of its positive predictions were false (precision=0.357). The resulting low F1-score (0.497) further emphasized the challenge posed by class imbalance.

The model's high recall demonstrates its strength in identifying diabetes cases, making it effective for screening purposes. However, the low precision for the positive class indicates a high false positive rate, which could lead to unnecessary anxiety or additional examination costs for non-diabetic individuals. Therefore, it is essential to adjust the decision threshold based on the specific application context and stakeholder needs. For instance, if minimizing false alarms and related costs is a priority, the threshold could be raised (e.g., to 0.7) to reduce false positives, though this would increase the risk of missed diagnoses. In addition to threshold tuning, resampling techniques (e.g. SMOTE) can also be employed to better address class imbalance and improve model performance.

2.2 Model optimization

To enhance the model's performance, a comprehensive hyperparameter optimization strategy was employed, focusing on the interaction effects between parameters rather than tuning them in isolation. As mentioned before, grid search combined with stratified k-fold cross-validation was conducted to empirically evaluate multiple parameter combinations. Learning rate (`eta`) and number of estimators (`n_estimators`) were jointly optimized to balance convergence speed and model stability. A smaller learning rate (e.g., `eta = 0.01`) was used to ensure gradual updates, paired with a higher number of estimators (`n_estimators = 50`) to maintain predictive power without overfitting. Tree depth (`max_depth`) and sampling strategies (`subsample`, `colsample_bytree`) were tuned to control model complexity. Deeper trees (`max_depth = 5`) offer higher capacity but risk overfitting; thus, subset sampling (`subsample = 0.8`) and column sampling (`colsample_bytree = 0.8`) were used to introduce randomness and improve generalization. All combinations were evaluated using metrics PR-AUC, which is especially effective for imbalanced classification tasks.

Additionally, the SMOTE strategy was applied enhance the model's ability to identify minority class cases by synthesizing minority class samples. To evaluate the capacity of the original model, the original imbalanced dataset (non-SMOTE) was retained and optimized by tuning the `scale_pos_weight` parameter to solve the class imbalance problem, ensuring sufficient attention to the positive class. Empirical results show that models trained on the original imbalanced dataset using the `scale_pos_weight` parameter slightly outperformed those trained on SMOTE-enhanced data, indicating that the algorithm-based approach was more effective than the sampling-based solution in this case.

Individual Component – kwan0569

1. Predictive model

1.1 Model Description

The prediction model I chose is CatBoost (Categorical Boosting). This is an ensemble learning method based on gradient boosting decision trees, specially designed to handle data containing a large number of categorical variables. The assumptions underlying of this model is to require a fractal, non-stationary and category-dependent relationship between the target variable and the features. And the model can capture these complex relationships through the iterative improvement method of multiple weak classifiers.

The strengths of this technique are as follows: Firstly, when using this model, there is no need to perform One-Hot or Label Encoding in advance. Secondly, because this model uses built-in category statistics and ranking improvement algorithms, it can reduce the risk of data leakage. Thirdly, it can automatically handle missing values and reduce the complexity of preprocessing. Fourthly, this model, through Ordered Boosting, has a strong overfitting control ability.

As for its limitations, they are also obvious. Compared with neural network systems, CatBoost has limited ability to process continuous high-dimensional features. And because this model belongs to the black-box model, its structure is somewhat unexplainable. Finally, the training speed of this model is very slow, especially during the process of parameter search. For example, when I used “GridSearchCV”, there was a waiting time of more than one hour.

Because in the diabetes health indicator dataset, there are multiple class variables with high cardinality, such as “Sex”, “Race”, “Smoker”, etc. However, using CatBoost can eliminate the complex class encoding preprocessing and can effectively alleviate the problem of positive and negative class imbalance by combining “class_weights”. Moreover, this model performs excellently in PR-AUC and Recall. Therefore, it indicates that the CatBoost model is suitable for studying the problems raised by our group and also for the characteristics of the dataset selected by our group.

1.2 Model Algorithm

As mentioned in 1.1, CatBoost is an ensemble learning algorithm based on gradient boosting decision trees. Its core idea is to correct the residuals of the previous round of the model by constantly training new weak learners (decision trees), thereby gradually approaching the true label. The core of its algorithm is "Ordered Boosting", in order to prevent data leakage in the calculation of class variables; “Target Statistics (CTR)” is to construct statistical features by using the combination of category frequency and target value for category features.

In CatBoost, the mathematical formulas used are:

- The overall optimization objective of CatBoost:

(1) CatBoost uses the Gradient boosting framework to optimize the loss function L in the form of:

$$F(x) = \sum_{t=1}^{T} \eta \cdot h_t(x)$$

$F(x)$: The prediction function of the final model. $h_t(x)$: The t tree. η: Learning rate. T: Total number of trees.

(2) In each round, minimize the gradient residual of the loss function:

$$r_i^t = -\frac{\partial L\big(y_i, F(x_i)\big)}{\partial F(x_i)}\Big|_{F=F^{(t-1)}}$$

(3) The training objective of CatBoost is:

$$min \sum_{i=1}^{n} L(y_i\,, F(x_i)) + \lambda \parallel F \parallel^2$$

- Category feature processing:

CatBoost does not perform One-Hot Encoding but calculates the target statistics of the category, namely CTR Encoding:

$$CTR(x_i) = \frac{\sum_{j<i} \parallel \left[x_j = x_i\right] \cdot y_j + a \cdot p}{\sum_{j<i} \parallel \left[x_j = x_i\right] + a}$$

a: Smoothing parameters (to prevent small sample fluctuations). p: Overall positive case probability. ‖: Indicate whether the function belongs to the same category.

- The loss function of CatBoost (Taking binary classification as an Example):

Log Loss:

$$L(y, \hat{y}) = -y log(\hat{y}) - (1 - y) log(1 - \hat{y})$$

It is applicable to binary classification tasks, such as my current diabetes prediction task.

- Regularization term (to prevent overfitting):

The complexity of leaf nodes is controlled through L2 regularization terms:

$$Objective = \sum_{i=1}^{n} L(y_i, F(x_i)) + \lambda \sum_{leaves} w^2$$

λ is l2_leaf_reg in CatBoost.

Screenshot of the pseudocode for CatBoost's hyperparameters:

```
Input: training set (X, y), cat_features list

Initialize model F₀(x) = 0

For t = 1 to T (iterations):
    Compute gradient residuals: rᵢ = ∂Loss(yᵢ, Fₜ₋₁(xᵢ))

    Estimate target statistics (CTR) for categorical features using ordered boosting

    Train a symmetric tree hₜ(x) on residuals

    Update model: Fₜ(x) = Fₜ₋₁(x) + η * hₜ(x)

Return final model: F_T(x)
```

1.3 Model Development

During the model development process, I first grouped the original data set of the first stage using the "groups" function for BMI. Subsequently, I calculated the category weights using group counting and smoothed them, generating three versions: namely "bmi_group_5", "bmi_group_10" and "bmi_group_20". These smooth weights are used to alleviate the problem of uneven BMI distribution. Then in the feature engineering, I added "bmi_group" as a derivative feature to the original data, and then used "RepeatedStratifiedKFold" for repeated stratified sampling (repeats=3, splits=5) for the evaluation of the stable model. This is a cross-validation strategy for category-imbalanced data. When facing data category imbalance, I handle it using "class_weights=[1, 6]". To make the CatBoost model pay more attention to the positive class (diabetic) and improve the Recall performance. It can be seen from the results of PR-AUC and recall rate that this strategy works well (As shown in Fig. 1.3.1).

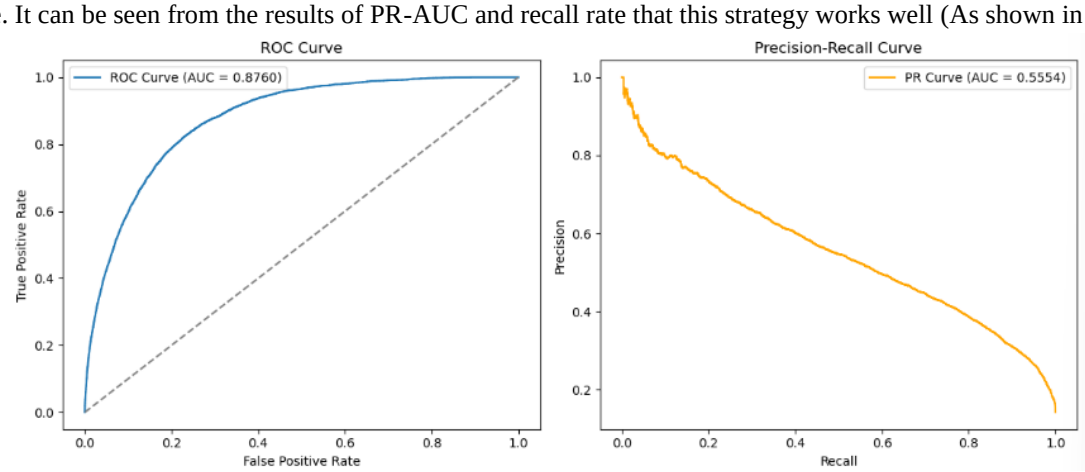


Fig. 1.3.1

In terms of the Python functions I use "CatBoostClassifier()" is used to construct the CatBoost model; "GridSearchCV()" is used for hyperparameter tuning (grid search); "fit()" is used for training the model; Use "predict()" for prediction; "predict_proba()" is to output the predicted probability (used to calculate PR-AUC); "classification_report()" is used to output the "precision", "recall", and "f1-score" metrics; "roc_auc_score()" and "average_precision_score()" are respectively used to calculate "ROC-AUC" and "PR-AUC". In the CatBoost hyperparameter, "depth" represents the maximum depth of each tree and is used to control the model complexity. "iterations" represents the total number of training rounds. "learning_rate" is the learning rate, which determines the extent of model update in each iteration. "L2_leaf_reg" is the L2 regularization coefficient, which controls the model complexity and prevents overfitting. "class_weights" is used to adjust the importance weights of

positive and negative classes. “cat_features” is used to tell the model which are categorical variables, and CatBoost will handle it automatically.

2. Model Evaluation and Optimization

2.1 Model Evaluation

```
              precision    recall  f1-score   support

           0       0.96      0.78      0.86     34072
           1       0.38      0.82      0.52      5644

    accuracy                           0.79     39716
   macro avg       0.67      0.80      0.69     39716
weighted avg       0.88      0.79      0.81     39716

ROC-AUC: 0.8811295345047441
PR-AUC : 0.5713785200634289
```

In the left figure, category 0 represents no diabetes. The precision (0.96) in the figure indicates that most of the predicted zeros are true zeros. Recall (0.78) indicates that 22% of the true zeros are misclassified. Category 1 indicates the presence of diabetes, with a relatively low Precision of 0.38, suggesting that many predictions of 1 are incorrect (with a high incidence of false positives). The Recall is relatively high, at 0.82, indicating that most of the true 1s have been successfully found. And the Accuracy is 0.79, which is relatively in line with expectations. The ROC-AUC is 0.881, indicating that the model has a strong ability to distinguish between 0 and 1, and the closer it is to 1, the better. The PR-AUC is 0.571, indicating that under imbalanced data, more attention is paid to the recognition effect of positive classes. F1-score is the harmonic mean of Precision and Recall, and it is an important indicator to measure whether the model can control the false alarm rate while maintaining the recall ability. Although the model has relatively low Precision, due to its high Recall, the F1-score has reached 0.52, which has strong practical value in the medical screening scenario (it is more willing to report more suspicious cases wrongly rather than miss diagnoses). The conclusion is that the model shows a strong discrimination ability, especially for the recognition of positive classes, the effect is remarkable. Although the precision is relatively low, the recall is high, indicating that the model pays more attention to "avoiding missed diagnoses as much as possible", which is more practical in medical screening scenarios. Furthermore, the model handles category variables excellently, automatically completes Target Statistics, and supports category sorting processing, avoiding the risk of data leakage. As for the potential areas for improvement, I think the decision threshold can be adjusted. For example, changing the default 0.5 to a higher one and using F1-score as the optimization objective for training might lead to more ideal results.

2.2 Model Optimisation

In model optimization, I used GridSearchCV to adjust the parameters of the hyperparameters. The target indicator is PR-AUC. The finally selected parameter combination is: “depth=8”, “iterations=300”, “l2_leaf_reg=3”, “ learning_rate=0.1”. The average PR-AUC of this combination reached 0.5688 in the 5 splits cross-validation (As shown in Fig. 2.2.1), which was superior to the performance under the default parameters: 0.5554 (As shown in Fig. 1.3.1). This indicates that parameter tuning effectively enhances the model's recognition ability for positive class samples (diabetes).

```
Fitting 5 folds for each of 36 candidates, totalling 180 fits
Best parameters: {'depth': 8, 'iterations': 300, 'l2_leaf_reg': 3, 'learning_rate': 0.1}
Best PR-AUC: 0.5687775687726603
```

Fig. 2.2.1

Individual Component – yzan0977

1. Predictive model

1.1 Model Description

In this assignment, the modelling technology I chose is LightGBM (Light Gradient-Boosting Machine), which is a distributed Gradient Boosting framework based on decision trees proposed by Microsoft. It supports tasks such as classification, regression and ranking, and the core goal lies in efficient training and high-precision prediction. Based on the traditional GBDT (Gradient Boosting Decision Tree), it introduces innovations such as histogram algorithm, leaf-first growth, gradient one-sided sampling (GOSS), and feature exclusive bunding (EFB), bringing significant training acceleration and memory optimization to large-scale and high-dimensional datasets. Meanwhile, maintain or improve the performance of the model.(Ke et al., 2017, p. 3147) (Contributors to Wikimedia projects, 2020)

This technology is based on the following core assumptions:

- **The Loss function is differentiable:** LightGBM depends on the gradients and second-order gradients (Hessian) of the model output. Therefore, the loss functions used must be differentiable.
- **The cumulative effectiveness of weak learners:** Although each decision tree is a weak learner, it can continuously correct residuals through iterative addition.
- **Sample independence and uniform distribution:** It is assumed that the training samples are independent of each other and consistent with the distribution of future test data.
- **Validity of gradient information:** GOSS believes that samples with large gradients contain more valuable information and can be retained. After downsampling with small gradient samples, the overall gradient distribution can still approximate the original distribution.

In the context of the diabetes dataset, the following critical evaluations of LightGBM can be made: On the diabetes dataset with 198,578 samples (28,220 positive cases, 170,357 negative cases, with a ratio of approximately 1:6), LightGBM alleviates the class imbalance by adjusting the sample weights, and its native histogram automatically processes numerical features by box without additional preprocessing. The leaf-priority splitting strategy can efficiently capture the nonlinear interaction of 8-10 dimensional features. However, it is prone to generating overly deep trees under a medium sample size, and regularization parameters need to be used to prevent overfitting. Although the global feature importance can be output, it is difficult to explain a single prediction intuitively. Local explanations can be generated by combining tools such as SHAP to enhance transparency and credibility.

Overall, for the diabetes dataset used this time, the advantages of using LightGBM lie in its high training efficiency and support for large-scale scaling, native support for imbalanced data, complex nonlinear modeling capabilities, and the ability to simplify numerical feature processing. However, the disadvantages lie in the possible risk of overfitting in the small feature dimension, insufficient interpretability, time-consuming hyperparameter tuning and high requirements for computing resources. Through the above discussion, it can be found that LightGBM has sufficient advantages in the research questions and dataset characteristics of this time, and the defects can be effectively improved through other methods or techniques.

1.2 Model Algorithm

LightGBM inherits the additive model idea of GBDT: each new tree fits the residual of the current model to minimize the loss function. Different from the traditional level-wise growth, LightGBM adopts the leaf-wise growth strategy, which can achieve lower training loss under the same number of leaf nodes. To accelerate the training of large-scale datasets, several core optimization techniques such as GOSS, EFB and Histogram Binning have also been introduced, significantly reducing the time and memory overhead.

- Mathematical principle:

Objective Function and Gradient boosting: GBDT constructs an approximate objective by performing a second-order Taylor expansion of the differentiable loss function $L(y_i, \hat{y}_i)$ at the current prediction $\hat{y}_i^{(t)}$:

$\sum_{i=1}^{n} L(y_i, \hat{y}_i^{(t)} + f_t(x_i)) \approx \sum_{i=1}^{n} [g_i f_t(x_i) + \frac{1}{2} h_i f_t(x_i)^2]$, Where g_i is the first Gradient and h_i is the second Hessian. The learning of the new tree f_t is to set a constant value w_j at each leaf node to maximize the Gain function:

$Gain = \frac{1}{2}[\frac{(\sum_{i\in I_L} g_i)^2}{\sum_{i\in I_L} h_i + \lambda} + \frac{(\sum_{i\in I_R} g_i)^2}{\sum_{i\in I_R} h_i + \lambda} - \frac{(\sum_{i\in I} g_i)^2}{\sum_{i\in I} h_i + \lambda}] - \gamma$, Among them, λ is the second-order regularization coefficient and γ is the minimum gain threshold.

- Main hyperparameters:

boosting_type: gbdt, dart, goss, rf, etc. (determine the core variant). **num_leaves:** The maximum number of leaf nodes, controlling the complexity of the model. **learning_rate:** Step size reduction. When combined with large num_leaves, a small value often needs to be set. **max_depth:** Tree depth limit, which helps prevent leaf preference from causing overfitting. **min_data_in_leaf, min_sum_hessian_in_leaf:** Lower limit the sum of the number of samples per leaf and Hessian to enhance regularization. **feature_fraction, bagging_fraction, bagging_freq:** Sub-sampling parameters of features and samples, similar to random forest sampling, to improve generalization. **lambda_l1, lambda_l2:** L1, L2 regularization, suppressing excessive leaf weights.

- Algorithm pseudocode:

```
LightGBM

Input: D={(xᵢ,yᵢ)}, T, η, λ, γ, num_leaves, max_depth

F ← 0
for t = 1 to T:
  compute gᵢ ← ∂L/∂F(xᵢ), hᵢ ← ∂²L/∂F(xᵢ)
  (opt) apply GOSS/EFB to {g,h}
  build tree f by leaf-wise splits on histograms until num_leaves/max_depth
  set each leaf weight w ← −Σg/ (Σh+λ)
  F ← F + η·f
return F
```

1.3 Model Development

In this study, I first standardized numerical characteristics such as BMI, age and income using **StandardScaler** for zero mean and unit variance to eliminate dimensional differences and accelerate model convergence. Subsequently, in the same pipeline, 95% variance principal components are retained through **PCA**(n_components=0.95), which not only reduces the feature dimension but also removes redundant noise; For category variables, **OrdinalEncoder** is used for encoding, and ColumnTransformer integrates the values with the category **preprocessing pipeline** to ensure consistent processing methods in each cross-validation. Based on the hierarchical cross-validation framework of 15 folds (3 repetitions ×5 folds), and using train_test_split to divide 20% of the validation set in each fold, I called **LGBMClassifier** to build the model. At the beginning, the hyperparameters I used in the model were based on experience. Referring to the commonly used hyperparameter combinations in other papers, then in the subsequent model optimization stage, I used grid search and Bayesian optimization to find better hyperparameters. And introduce strategies such as early stop to monitor the AUC performance of the training and validation sets in real time and prevent overfitting; After the training is completed, the threshold scan of the positive class probability of the validation set is performed through precision_recall_curve, and the global optimal classification threshold is selected based on the principle of F1 maximization. Finally, calculate and report accuracy_score, roc_auc_score, f1_score and average_precision_score on the test set, and summarize the average results of all fractures. It is used to evaluate the stability and generalization ability of the model under the conditions of unbalanced and nonlinear interaction characteristics. The function is called **use_lightgbm** in my code.

2. Model Evaluation and Optimization

2.1 Model Evaluation

In binary classification, the precision rate measures the accuracy of the predicted positive examples, the recall rate measures the coverage of the actual positive examples, and F1 is the combination of the two. ROC-AUC reflects the overall discrimination ability of the model under each threshold, while PR-AUC focuses on the performance of minority classes. In this study, I used these five indicators to evaluate the model.

Overall, this model achieved a relatively good overall discriminative ability of ROC-AUC=0.8420 under the background of positive and negative sample imbalance (approximately 6:1), but PR-AUC=0.4634 was significantly lower than ROC-AUC, reflecting the limited accuracy - recall trade-off in the prediction of a few classes (diabetes). Specifically, the prediction performance for the non-diabetic population (class 0) is excellent, with an accuracy rate of 0.931 and a recall rate of 0.862. However, for diabetic patients (class 1), it can only achieve an accuracy rate of 0.423 and a recall rate of 0.613, meaning that

although approximately 61% of the real patients can be captured, it is accompanied by a relatively high false alarm rate. This led to the F1 score being only 0.5006. Combining the noise in the dataset features (such as skinfold thickness, insulin, etc., deletions or outliers) and the complex interaction of lifestyle indicators, this result indicates that the model is biased towards identifying healthy individuals but still has insufficient sensitivity to disease signals. To enhance the model's detection ability for diabetic patients, emphasis should be placed on strengthening learning on minority class samples, such as introducing cost-sensitive training or improving the sampling strategy to alleviate class imbalance; Further handle anomalies and missing values in the feature engineering stage; And combine interpretability tools (such as SHAP) to verify and optimize key features to enhance the clinical interpretability and reliability of the model.

2.2 Model Optimisation

In my code, multiple optimization methods have been applied to improve the performance of the LightGBM model:

- **Unified assembly line preprocessing:** The numerical features are first standardized by StandardScaler, then dimensionally reduced by PCA(n_components=0.95), and combined with OrdinalEncoder to encode the category features. The preprocessing of all columns is completed at one time through ColumnTransformer. It not only ensures data standardization but also eliminates redundant noise.
- **Repeated hierarchical cross-validation:** A hierarchical CV with 3 repetitions ×5 splits for a total of 15 splits is adopted, and within each split, the validation set is further divided using train_test_split to ensure a consistent ratio of positive and negative samples, while providing independent data for early stop and threshold calibration.
- **Dynamic sample weight:** Before each training, calculate scale_pos_weight=neg/pos based on the number of positive and negative samples of the current training subset, allowing the model to automatically focus on minority classes when constructing the gain.
- **Early stop and training records:** Inject the early_stopping(stopping_rounds=20), log_evaluation(period=0) and record_evaluation(evals_result) callbacks in.fit() The training is stopped after several consecutive rounds without improvement on the validation set AUC, and the index curve of each round is saved for analyzing the learning dynamics.
- **Global threshold calibration:** After the training is completed, aggregate the verification probabilities of all curves, scan the optimal threshold using the precision_recall_curve and F1 formula, and apply it to the prediction of the test set to replace the default 0.5 and further balance the precision rate and recall rate.
- **Hyperparameter tuning interface:** I additionally implemented **tuning_by_grid_search** (grid search based on ParameterGrid + cross_val_score) and **tuning_by_bayes** (Bayesian optimization of Optuna). It provides a complete tool for the subsequent automated search of the optimal combination and the improvement of model performance.

Group Component 2

1. Discussion

The research question posed in this study is: Is it possible to effectively predict whether an individual has diabetes based on demographic information, health status, and lifestyle? To address this question, we employed three mainstream gradient boosting decision tree models: XGBoost, CatBoost, and LightGBM. While all three models share the core idea of gradually fitting the residuals and iteratively combining weak learners into a strong ensemble, there are essential differences in implementation details and optimization strategies, and these differences directly affect their performance, computational efficiency, memory consumption, and ease of use.

Theoretically, XGBoost adopts level-wise tree growth strategy, expanding all leaf nodes at the same depth in each iteration. This generally results in more balanced structure, which can enhance accuracy and make it perform stable in most evaluation metrics, but comes at the cost of increased training time and memory usage On the other hand, LightGBM is based on histogram-based gradient boosting and adopts a leaf-wise growth strategy, where in each iteration only the leaf with the highest gain is expanded, which greatly improves the speed and reduces memory consumption. However, this aggressive growth approach can lead to overfitting, particularly on small or imbalanced datasets. If not properly tuned, LightGBM may underperform in tasks where achieving high recall is critical. Since the one of primary objective of this research is to minimize the rate of missed diagnoses by maximizing recall, LightGBM is less likely to produce optimal results in this context. CatBoost introduces a symmetric tree structure, where all leaves at a given level follow the same split criteria, making the training more stable and having strong generalization ability. It also demonstrates significant advantages in natively processing categorical variables. With its core innovations in ordered boosting and categorical boosting, CatBoost applies a permutation-driven technique that effectively reduces prediction bias and mitigates the risk of overfitting, even without extensive hyperparameter tuning. These strengths are particularly beneficial when working with noisy data or datasets that exhibit class imbalance. However, one limitation is that compared with XGBoost and LightGBM, CatBoost typically requires longer training time since it is a kind of permutation-driven technique.

In practical applications, LightGBM demonstrates clear advantages. For the dataset used in this study, which contains over 200,000 samples, it achieves significantly faster training speeds compared to the other two models. More importantly, CatBoost offers several additional strengths, particularly in handling categorical features. It provides native support for categorical variables and employs a unique ordered encoding technique that effectively avoid data leakage and simplify the data preprocessing. This makes CatBoost especially well-suited for current datasets with ordinal variables or a high proportion of categorical features. Furthermore, it demonstrates greater robustness when dealing with class-imbalanced data, which closely aligns with the characteristics of our dataset and the demands of the research question. In contrast, XGBoost performs less favourably in both training speed and predictive performance compared to LightGBM and CatBoost, making it a less optimal choice for this study.

Supported by multiple quantitative empirical evidence, CatBoost consistently demonstrated superior results in the task of diabetes prediction. It achieved the highest ROC-AUC (0.88) and PR-AUC (0.57) scores among the three models. Although LightGBM showed the fastest training speed, it delivered the weakest predictive performance, with a ROC-AUC of 0.84 and a PR-AUC of 0.46. XGBoost performed moderately across all metrics, without any significant strengths or weaknesses. Due to the class imbalance in the dataset, where positive cases represent individuals with diabetes, particular attention was given to the models' ability to identify the positive class. In this regard, CatBoost again proved to be the most effective model. It achieved the highest recall of 0.82, indicating a strong ability to detect true positive cases. Although its precision of 0.38 was slightly lower than that of LightGBM (0.46), its higher F1-score of 0.52 suggests a better balance between minimizing false negatives and false positives. This balance is especially important in healthcare-related applications such as diabetes prediction, because missing a positive case (low recall score) can have serious consequences.

	Model	ROC-AUC	PR-AUC	Precision_0	Recall_0	F1_0	Precision_1	Recall_1	F1_1
0	LightGBM	0.84	0.46	0.93	0.86	0.90	0.42	0.61	0.50
1	CatBoost	0.88	0.57	0.96	0.78	0.86	0.38	0.82	0.52
2	XGBoost	0.86	0.50	0.96	0.76	0.85	0.36	0.82	0.50

Based on the above quantitative and qualitative analyses, including theoretical insights and practical comparisons, it is clear that different models have different strengths in addressing research question. For example, LightGBM offers the fastest training

speed in scenarios involving large datasets. XGBoost provides a more stable and balanced performance, making it a reliable and conservative choice. CatBoost is good at handling categorical features, which aligns well with the characteristics of real-world medical datasets. Meanwhile, the primary objective of this research is to accurately identify individuals at risk of diabetes. In such medical contexts, missed diagnoses (false negatives) may have more serious consequences than false positives. Considering this situation, recall and PR-AUC are more important than overall precision because we prioritize capturing as many truly positive cases as possible. From this perspective, the superior recall and PR-AUC of CatBoost make it the most suitable model. In conclusion, model selection should align not only with evaluation metrics but also with the specific nature of the research question. Although each of the three models demonstrates distinct advantages, CatBoost offers the most balanced combination of predictive performance, ease of use, and suitability for domain-specific applications, which makes it optimal predictive model for this research.

2. Conclusion

Based on the evaluation results of the three prediction models CatBoost, LightGBM and XGBoost, we conclude that all three models perform well in predicting individuals without diabetes (Class 0). Therefore, in relation to our research question, these models are effective for identifying non-diabetic individuals. However, for individuals with diabetes (Class 1), the predictive performance is limited due to the impact of class imbalance, indicating that the current models lack sufficient validity in this area.

Based on the comparison between theoretical and practical insights in the Discussion part, CatBoost is identified as the optimal predictive model for the current research. In disease prediction tasks, the selection of model is not merely a matter of numerical performance; it also has direct implications for public health strategies and clinical decision-making. Choosing CatBoost can help reduce both missed diagnoses rates and misdiagnosis rates. Its unique innovations, including Categorical Boosting and Ordered Boosting, provide strong support for medical research and data modelling, enhancing the feasibility and effectiveness of machine learning applications in clinical practice. Based on the above research, it is evident that future studies should focus on optimizing data collection strategies. For example, attention should be paid to the distribution of sample categories in order to reduce class imbalance, and efforts should be made to minimize selection bias. In the current dataset, for instance, the overall BMI levels are relatively high, with the majority of subjects falling into the obese category, which may affect the generalizability of the model.

At the technical level, further advancements are needed in handling class-imbalanced data. This issue is not only prevalent in medical datasets but is also common in other domains, such as financial risk detection, where the minority class, including fraudulent cases, plays a critical role despite its low occurrence. Given that class imbalance is a widespread challenge across many real-world applications, it is essential to develop robust and effective solutions to address this problem.

Finally, medical-related research requires interdisciplinary collaboration. It is necessary to combine the actual needs and knowledge in the medical field, adjust the "screening" and "diagnosis" thresholds of the model output, and balance the accuracy and recall rate at Class 1. Moreover, we can introduce medical indicators, physical health data, etc. to enhance the model's ability to identify the characteristics of diabetes. Because incorporating domain expertise from medical professionals can improve both the interpretability and practical relevance of predictive models, thereby enhancing their applicability in clinical decision-making.

To sum up, the current model has certain practicality in predicting "no diabetic" individuals. However, if effective identification of "diabetic patients" is to be achieved, further improvements need to be made in the matching of data, methods and business requirements to build a prediction system that can be used in actual medical scenarios.

Reference:

Ada's Medical Knowledge Team. (2025, February 7). Diabetes.Ada Health. https://ada.com/conditions/diabetes/

Chen, T. (2014, October 22). *Introduction to boosted trees*. New Jersey Institute of Technology. https://web.njit.edu/~usman/courses/cs675_spring20/BoostedTree.pdf

Contributors to Wikimedia projects. (2020, August 8). LightGBM - Wikipedia. Wikipedia, the free encyclopedia. https://en.wikipedia.org/wiki/LightGBM

International Diabetes Federation. (2025). IDF Diabetes Atlas (11th ed.). International Diabetes Federation. https://diabetesatlas.org/resources/idf-diabetes-atlas-2025/

Kolli, A. (2024, February 14). Understanding CatBoost: The gradient boosting algorithm for categorical data. Medium. https://aravindkolli.medium.com/understanding-catboost-the-gradient-boosting-algorithm-for-categorical-data-73ddb200895d

Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T.-Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. In I. Guyon, U. von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, & R. Garnett (Eds.), Advances in Neural Information Processing Systems (Vol. 30, pp. 3146–3154). Curran Associates, Inc. https://proceedings.neurips.cc/paper/2017/file/6449f44a102fde848669bdd9eb6b76fa-Paper.pdf

Microstrong. (2019, December 23). *In-depth understanding of XGBoost*. Zhihu Column. https://zhuanlan.zhihu.com/p/83901304

NSW Health. (2023, May 10). Body Mass Index (BMI) calculator. Healthy Eating Active Living. https://www.health.nsw.gov.au/heal/Pages/bmi.aspx

Transfer G. (2022, March 24). *Common metrics explained for binary classification problems (Recall, Precision, Specificity, etc.)*. Zhihu Column. https://zhuanlan.zhihu.com/p/456930366

XGBoost Development Team. (2024, June 20). *XGBoost 2.1.0 release notes*. https://xgboost.readthedocs.io/en/latest/changes/v2.1.0.html#jun-20

XGBoost Development Team. (n.d.). *XGBoost parameters*. https://xgboost.readthedocs.io/en/stable/parameter.html

Appendix

```
Classification result:
               precision    recall  f1-score   support

           0      0.961     0.756     0.847     34072
           1      0.357     0.817     0.497      5644

    accuracy                          0.765     39716
   macro avg      0.659     0.787     0.672     39716
weighted avg      0.876     0.765     0.797     39716

ROC-AUC on test dataset: 0.8633048424031887
PR-AUC on test dataset: 0.5031857358692615
```

Figure II - Classification Result of XGBoost Model